Assignment: SDD, BDD, and TDD in AI-Assisted Software Development

Student Information

- Name: 滕以璿 Vivienne
- Student ID: 1113516

- Course: AI-Assisted Software Development
- Date: 2026/05/31

---

1. Introduction

In the contemporary era of software engineering, AI-assisted development tools (such as GitHub Copilot, ChatGPT, and Claude) have radically transformed how code is produced. While these AI tools possess the capability to generate massive volumes of source code within seconds, they lack intrinsic awareness of a developer's real, nuanced intentions. AI operates purely on patterns learned from data; hence, the quality of its output is fundamentally limited by the precision of its inputs.

To prevent AI from generating irrelevant, buggy, or misaligned code, developers must shift their focus from manual syntax writing to precise requirement engineering. This is where Specification-Driven Development (SDD), Behavior-Driven Development (BDD), and TDD (Test-Driven Development) become indispensable. These three methodologies provide structured frameworks—ranging from high-level constraints to exact system behaviors and verification boundaries. By feeding these formal structures into AI tools, developers can establish an unambiguous "source of truth," minimizing prompt engineering ambiguities and ensuring that the AI-generated software artifacts are highly accurate, reliable, and strictly aligned with human expectations.

---

2. Definition of SDD

Specification-Driven Development (SDD) is a methodology where the development process is governed and guided by a formal, structured software specification document before any implementation begins. Instead of diving straight into coding, developers first meticulously outline what the system is supposed to achieve.

In the context of AI-assisted development, SDD serves as the structural blueprint for the AI. A proper SDD specification explicitly details the following core components:

- **Goal:** The ultimate problem the software intends to solve.
- **Functional Requirements:** The specific capabilities, actions, and services the software must execute.
- **Input:** The exact data types, structures, and ranges the system will receive.
- **Output:** The expected format, type, and distribution of the data produced by the system.
- **Constraints:** Technical limitations, environmental rules, or mathematical boundaries that the system must strictly adhere to.
- **Acceptance Criteria:** A set of formal conditions and predefined benchmarks used by stakeholders to verify whether a system's implementation is fully correct and complete.

---

3. SDD: Student Grade Calculator

3.1 Goal

The primary purpose of the Student Grade Calculator is to automate the evaluation of academic performance. It ingests individual scores from multiple course components, applies a precise mathematical weighted formula to compute a consolidated final numerical score, and subsequently maps this score to a standard descriptive letter grade to provide a structured academic assessment.

3.2 Functional Requirements

- **Score Ingestion:** The system must accept exactly four independent numerical scores corresponding to Assignments, Midterm Exam, Final Exam, and Project.
- **Weighted Compilation:** The system must execute a weighted arithmetic calculation using predetermined weights for each component to derive a single consolidated Final Score.
- **Precision Management:** The system must automatically round the calculated Final Score to exactly one decimal place before any grade mapping occurs.
- **Letter Grade Mapping:** The system must evaluate the rounded Final Score against a standardized, non-overlapping grading scale to assign the appropriate Letter Grade.
- **Robust Error Handling:** The system must validate all inputs prior to calculation and trigger an explicit, informative error message if any score violates established boundary ranges.

3.3 Input

The system requires four separate numeric parameters representing student performance. Each parameter must be a floating-point number or integer within the range of 0.0 to 100.0 inclusive:

- `Assignment Score` (Weight: 30%)
- `Midterm Exam Score` (Weight: 20%)
- `Final Exam Score` (Weight: 30%)
- `Project Score` (Weight: 20%)

3.4 Output

Upon a successful transaction, the system returns a structured response containing:

- `Final Score`: A rounded floating-point number representing the total weighted average (e.g., 85.4).
- `Letter Grade`: A single character string mapping representing the grade band (`A`, `B`, `C`, `D`, or `F`).

If validation fails, the output must strictly consist of:

- `Error Message`: A clear text alert indicating which input component failed verification and why.

3.5 Grade Rules

The final letter grade is strictly mapped according to the following mutually exclusive mathematical intervals after the final score has been rounded to one decimal place:

- **A:** $90.0 \leq \text{Final Score} \leq 100.0$
- **B:** $80.0 \leq \text{Final Score} < 90.0$
- **C:** $70.0 \leq \text{Final Score} < 80.0$

- **D:** $60.0 \leq \text{Final Score} < 70.0$
- **F:** $\text{Final Score} < 60.0$

3.6 Acceptance Criteria

- **[Original Criterion 1 - Exact Weighting and Rounding Verification]:** The calculated Final Score must strictly equal the exact mathematical sum of $0.30 \times \text{Assignment} + 0.20 \times \text{Midterm} + 0.30 \times \text{Final} + 0.20 \times \text{Project}$. The value must be rounded using standard half-up rounding to one decimal place, ensuring that an intermediate raw calculation of 89.95 becomes 90.0 (Grade A), while 89.94 becomes 89.9 (Grade B).
- **[Original Criterion 2 - Comprehensive Input Range Validation]:** The system must reject the entire operation if *any* single input score falls outside the strict boundary of $[0.0, 100.0]$. For instance, if a user attempts to input an assignment score of -5.0 or a project score of 101.5, the system must abort execution, refuse to calculate a final score, and throw a specific validation exception error.

---

4. Definition of BDD

Behavior-Driven Development (BDD) is an agile software development methodology that evolved from TDD, focusing on establishing a shared understanding of software requirements through collaborative, real-world behavioral scenarios. It bridges the communication gap between technical developers, QA engineers, and non-technical business stakeholders by describing software requirements in natural language.

BDD scenarios are written using a specialized, structured syntax called Gherkin, which follows the **Given-When-Then** format:

- **Given:** Establishes the initial state, preconditions, or context of the system.
- **When:** Specifies the precise action, event, or trigger executed by the user or system.
- **Then:** Outlines the expected observable outcome, post-conditions, or consequences of that action.

In the AI era, BDD is extraordinarily valuable because it translates abstract human operational expectations into concrete, sequential behavioral patterns. When an AI coding assistant is provided with Gherkin scenarios, it gains an explicit contextual framework that prevents it from misinterpreting how an end-user will interact with the system, leading to highly accurate UI flow and functional logic generation.

---

5. BDD: Student Grade Calculator

Scenario 1: Successfully mapping a balanced performance to a Grade B

```
Scenario: Student achieves a standard grade B with consistent scores
  Given the assignment score is entered as 85.0
  And the midterm exam score is entered as 78.0
  And the final exam score is entered as 82.0
  And the project score is entered as 88.0
  When the system processes and compiles the final grade statistics
  Then the compiled final score output should be displayed as 83.3
  And the resulting letter grade must be explicitly marked as B
```